Lab 9: CI/CD Fundamentals with GitHub Actions

Lab Summary

In this lab, you will implement CI/CD pipelines using GitHub Actions to deploy AWS resources. You will deploy resources using AWS CLI and automate deployments using workflow YAML files.

Prerequisites

- AWS account configured
- AWS CLI installed and configured (`aws configure`)
- Git installed
- GitHub account
- Python installed
- IAM permissions for Lambda, Glue, ECS, ECR, S3
- Existing S3 bucket

Steps to Complete Lab

Task 1 — Deploy Lambda Using AWS CLI

Step 1 — Navigate to Lambda Folder

```
cd lambda-functions
```

Step 2 — Zip Lambda Script

👉 Windows PowerShell

```
Compress-Archive -Path lambda_function.py -DestinationPath fu
nction.zip
```

👉 Linux / Mac

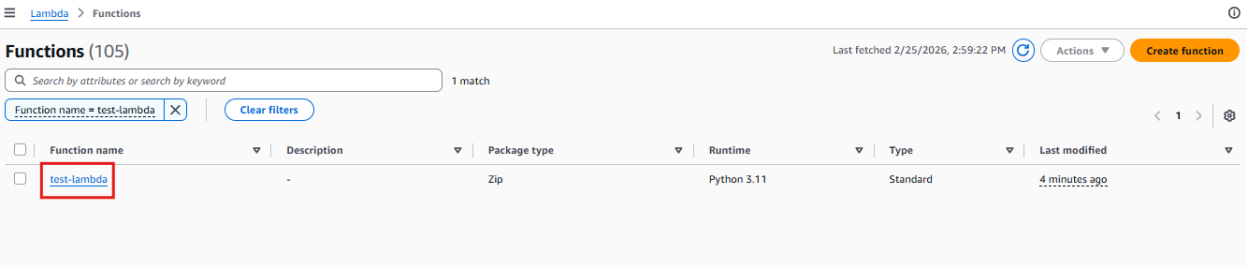
```
zip function.zip lambda_function.py
```

Step 3 — Create Lambda Function

```
aws lambda create-function \
--function-name test-lambda \
--runtime python3.11 \
--role <IAM_ROLE_ARN> \
--handler lambda_function.lambda_handler \
--zip-file fileb://function.zip
```

Step 4 — Verify Deployment

Go to AWS Console → Lambda → Confirm function exists



Task 2 — Deploy Glue Job Using AWS CLI

Step 1 — Upload Script to S3

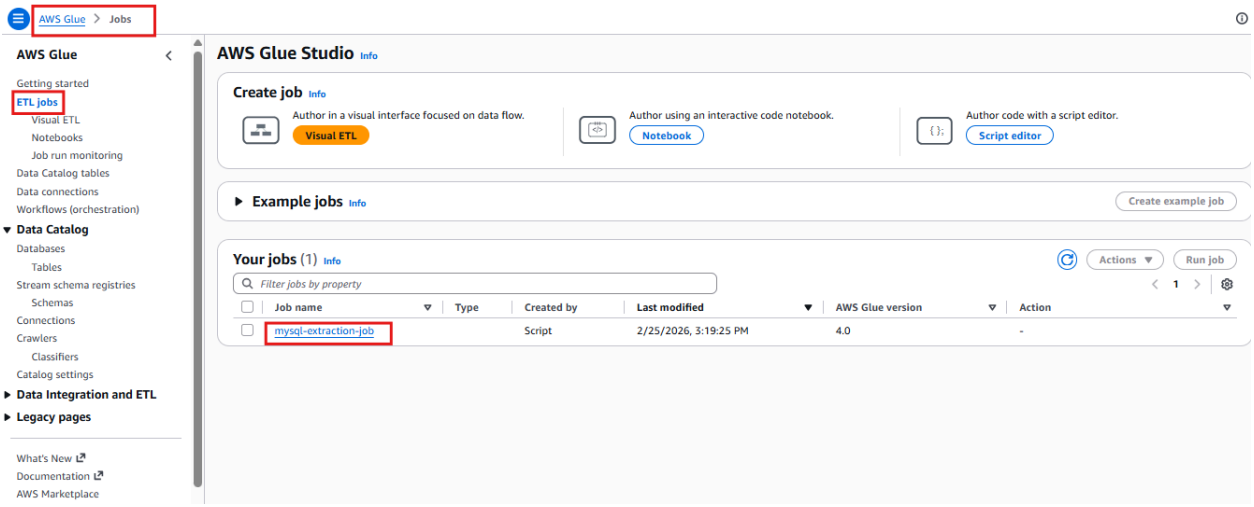
```
aws s3 cp mysql_extraction.py s3://<bucket-name>/glue-script
s/
```

Step 2 — Create Glue Job

```
aws glue create-job \
--name mysql-extraction-job \
--role <GLUE_ROLE_ARN> \
--command Name=pythonshell,ScriptLocation=s3://<bucket-name>/
glue-scripts/mysql_extraction.py,PythonVersion=3
```

Step 3 — Verify Job

AWS Console → Glue → ETL Jobs → Check job exists



Task 3 — Create GitHub Repository

Step 1 — Create Repo

GitHub → New repository

Upload:

- Glue script:
- Test scripts:

Step 2 — Add Workflow Directory

Create path:

```
.github/workflows/workflow.yml
```

Step 3 — Add GitHub Secrets

Go to Repository → Settings → Secrets → Actions

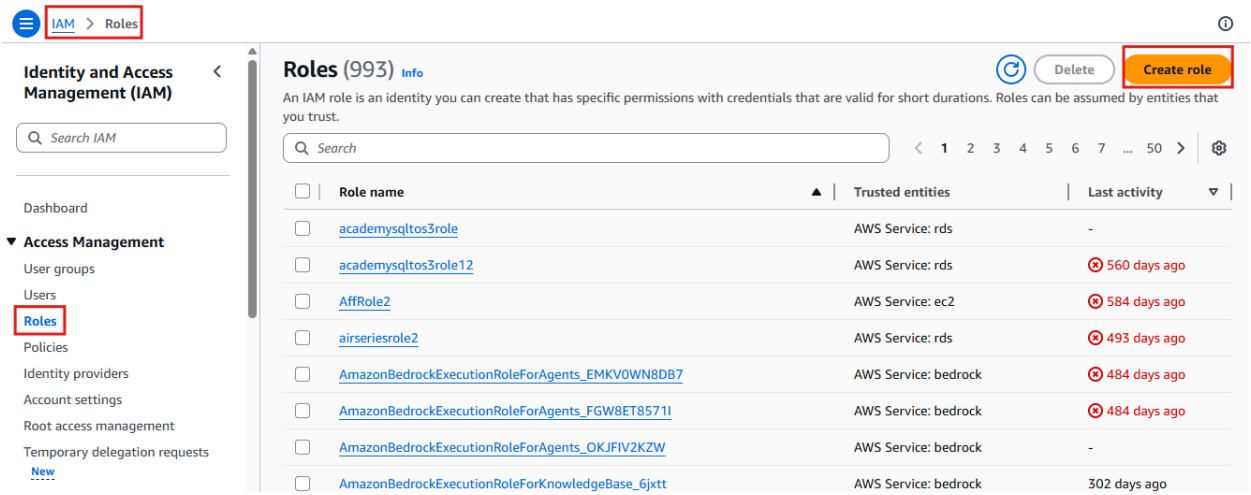
Add:

- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY

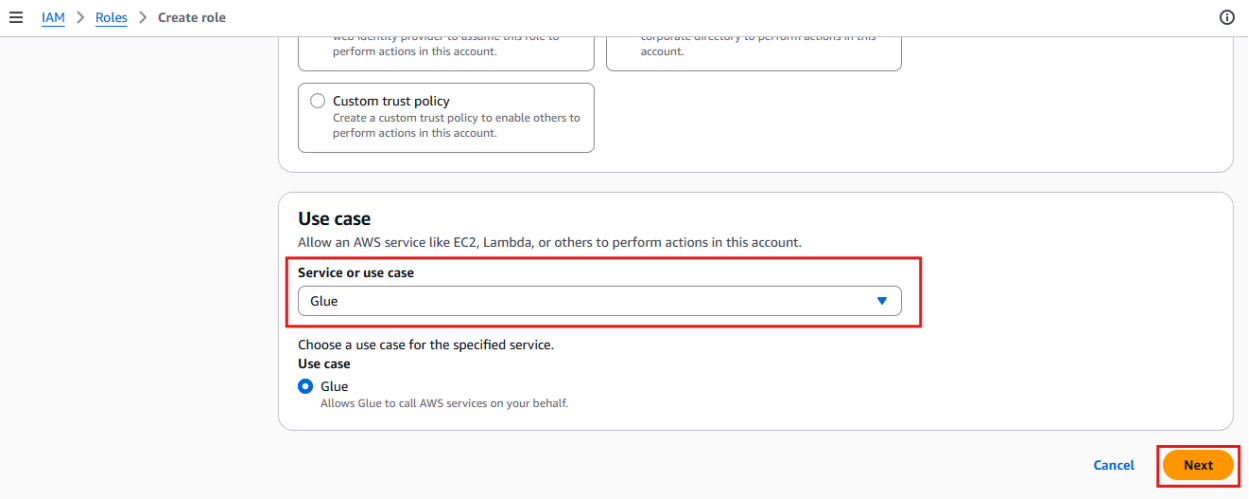
Task 4 — Create IAM Role for GitHub Actions

Step 1 — Create Role

AWS Console → IAM → Roles → Create Role



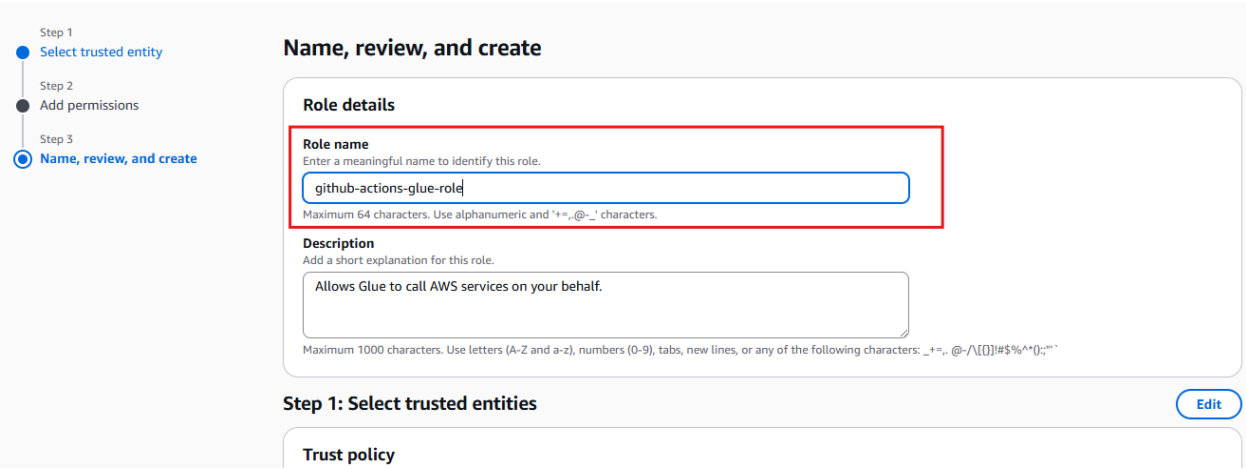
Select service → Glue



Attach permissions:

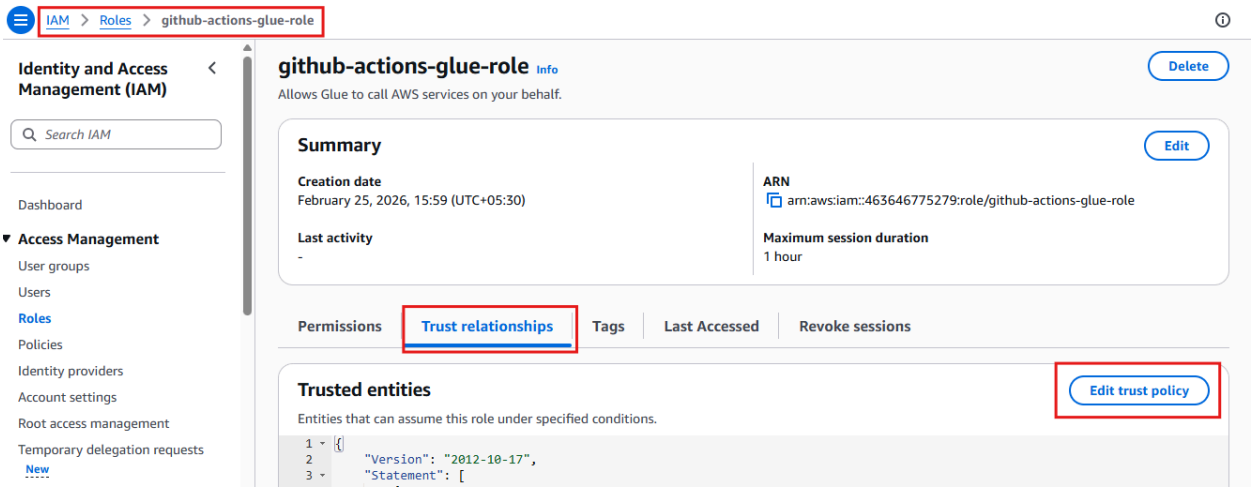
- GlueConsoleFullAccess
- S3FullAccess

Name the role —→ Click create role



Step 2 — Edit Trust Policy

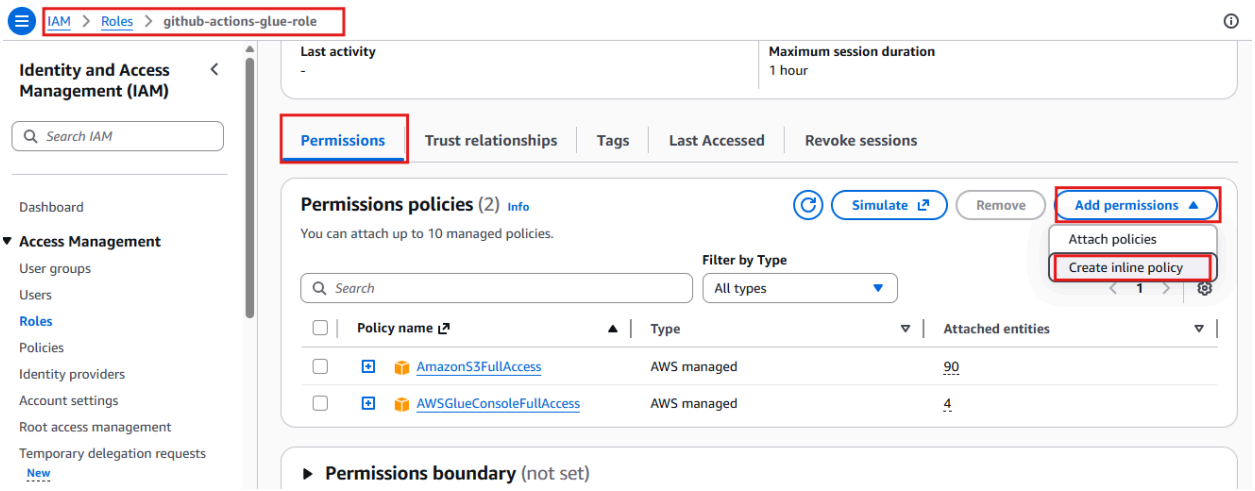
Click view role —→ Click on Trust relationship —→ Click on Edit trust policy



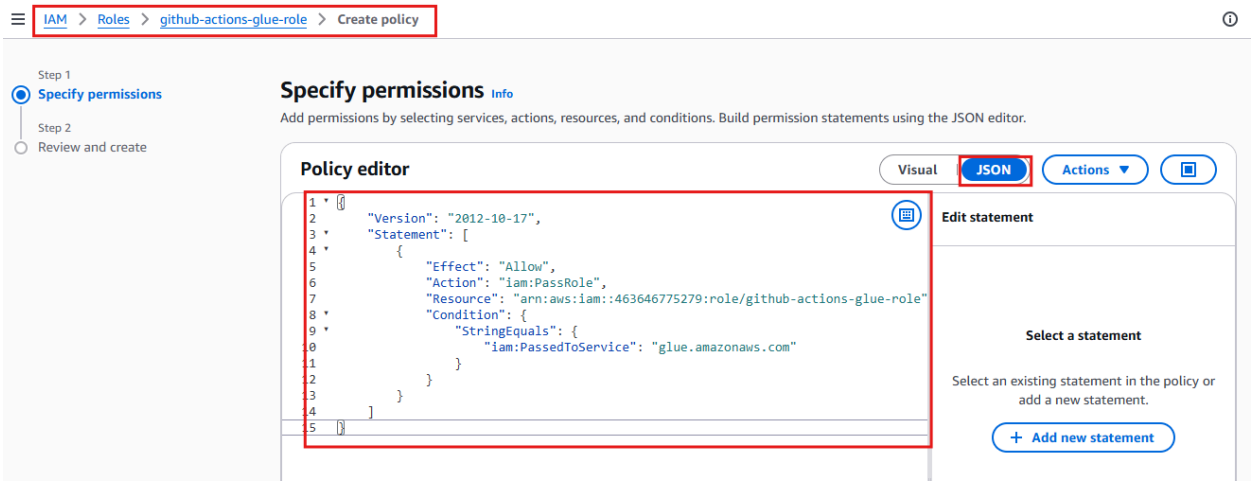
Replace with *glue_iam_trust_relationship.json* code

Update account number and GitHub username —→ Click update policy

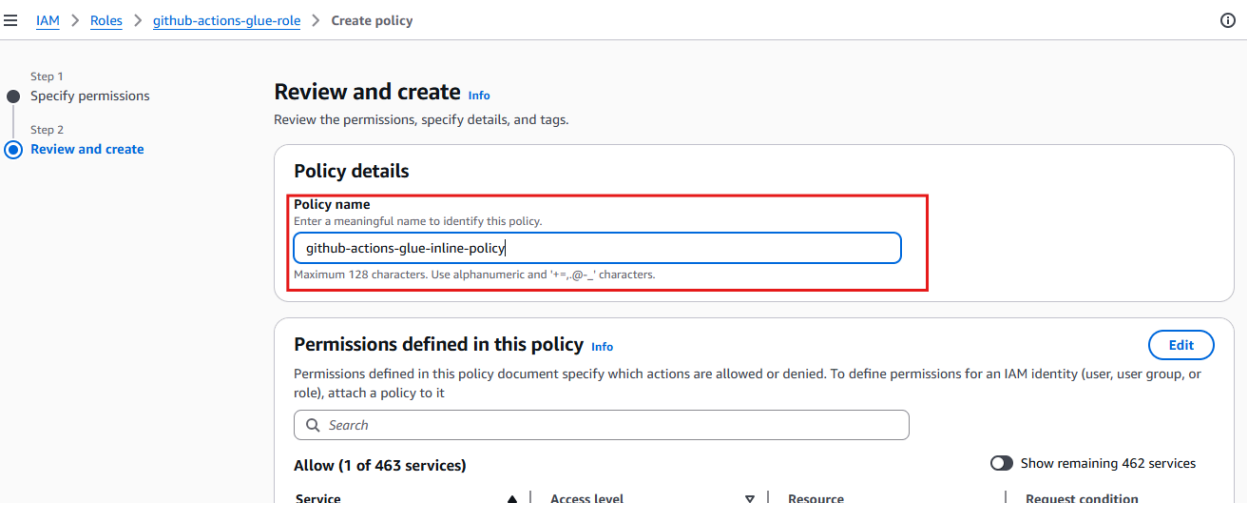
Add permission —→ create inline policy



Replace code from *glue-iam-inline-policy.json* —→ Replace role ARN with yours

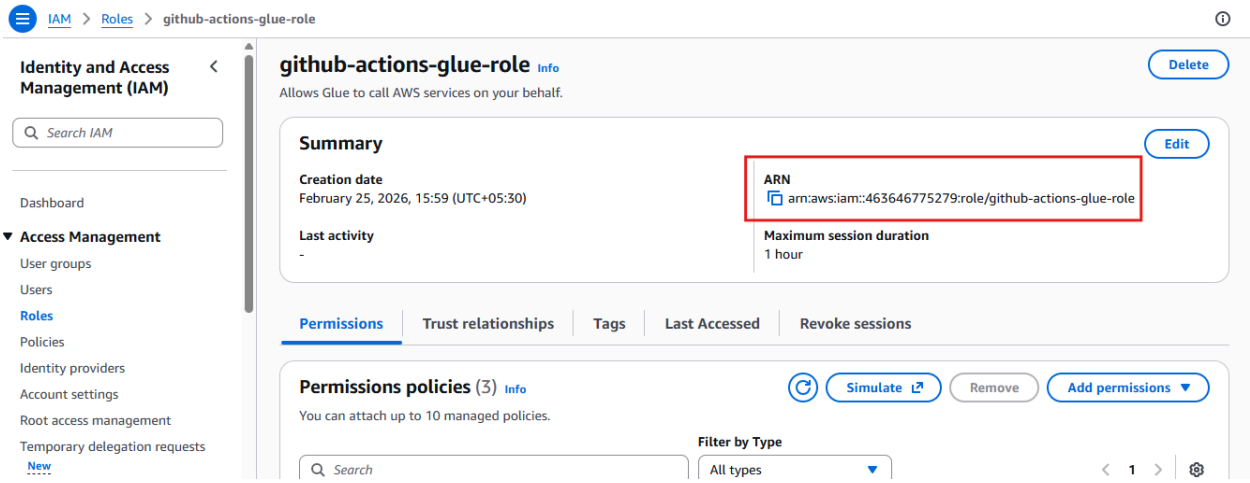


Name the policy —→ Scroll down —→ Click Create policy



Step 3 — Copy Role ARN

Copy the role ARN —→ Paste into workflow.yml file for github-actions-role ARN



Paste the while workflow.yml code to *.github/workflows/workflow.yml file*

Task 5 — Create Workflow YAML for Glue Deployment

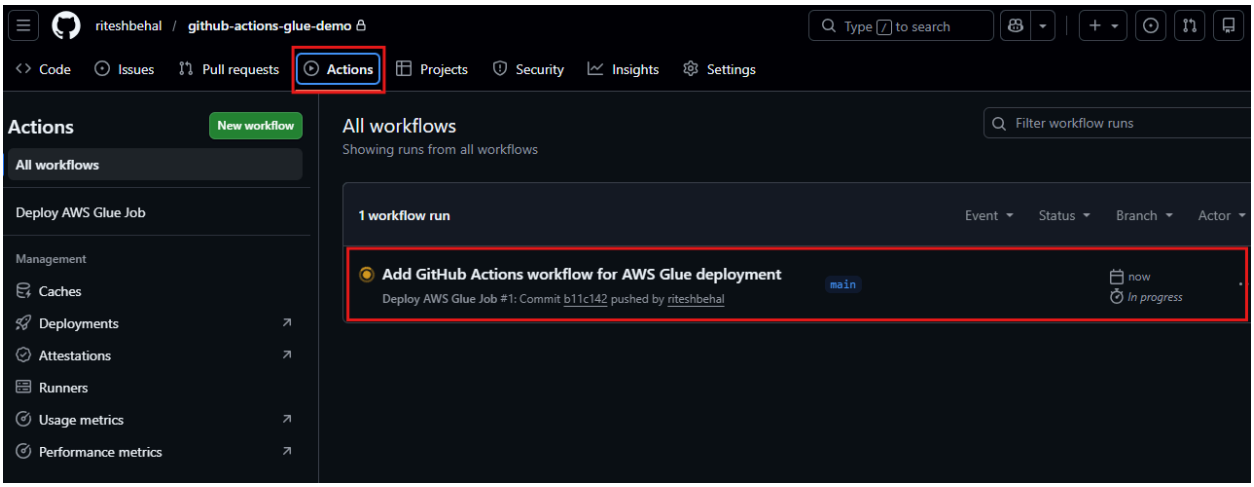
Step 1 — Add Workflow File

```
name: Deploy Glue Job

on:
  push:
```

```
    branches: [ main ]

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Install dependencies
        run: pip install awscli boto3

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v2
        with:
          aws-region: ap-south-1
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCES
S_KEY }}

      - name: Upload script to S3
        run: aws s3 cp mysql_extraction.py s3://<bucket-name
>/glue-scripts/

      - name: Deploy Glue job
        run: aws glue create-job --name mysql-extraction-job
--role <ROLE_ARN> --command Name=pythonshell,ScriptLocation=s
3://<bucket-name>/glue-scripts/mysql_extraction.py,PythonVers
ion=3
```

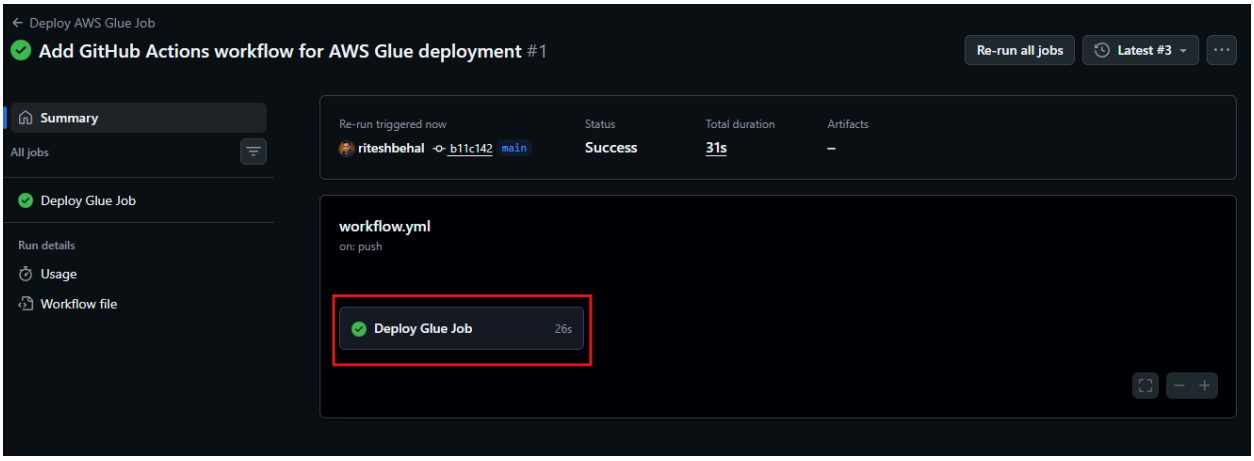
Step 2 — Commit Changes

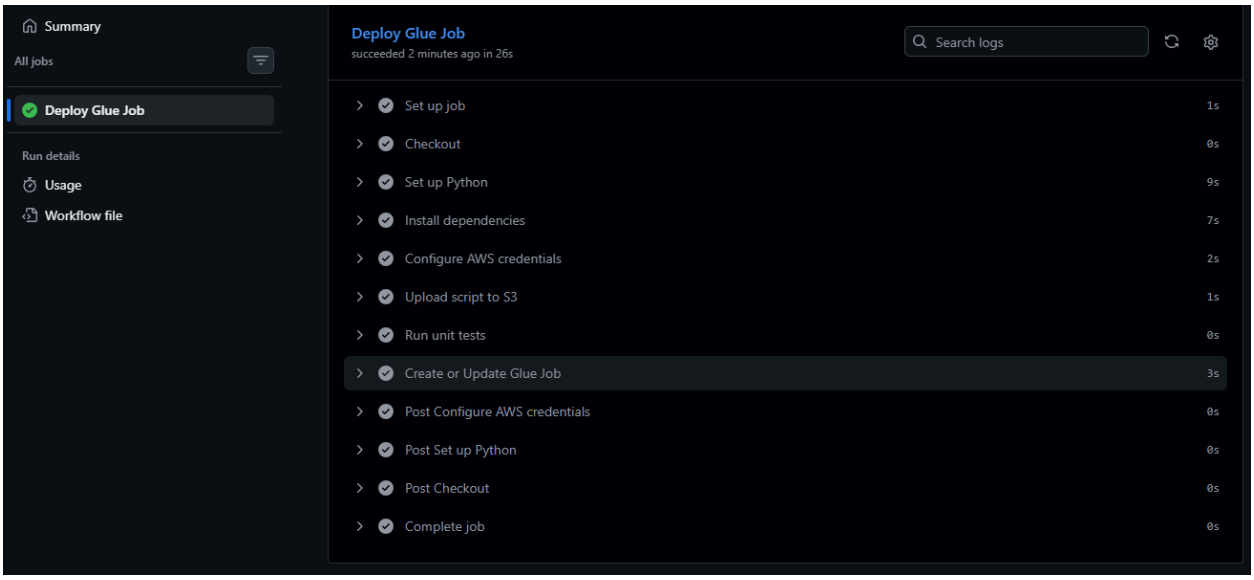
Push to GitHub → Workflow starts automatically



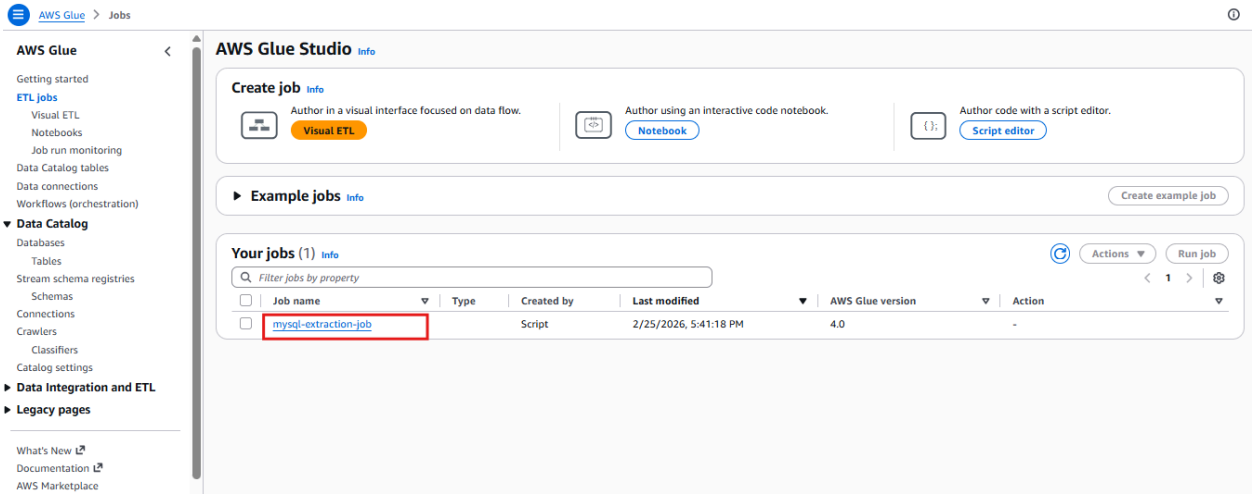
Step 3 — Verify Deployment

GitHub → Actions → Check logs





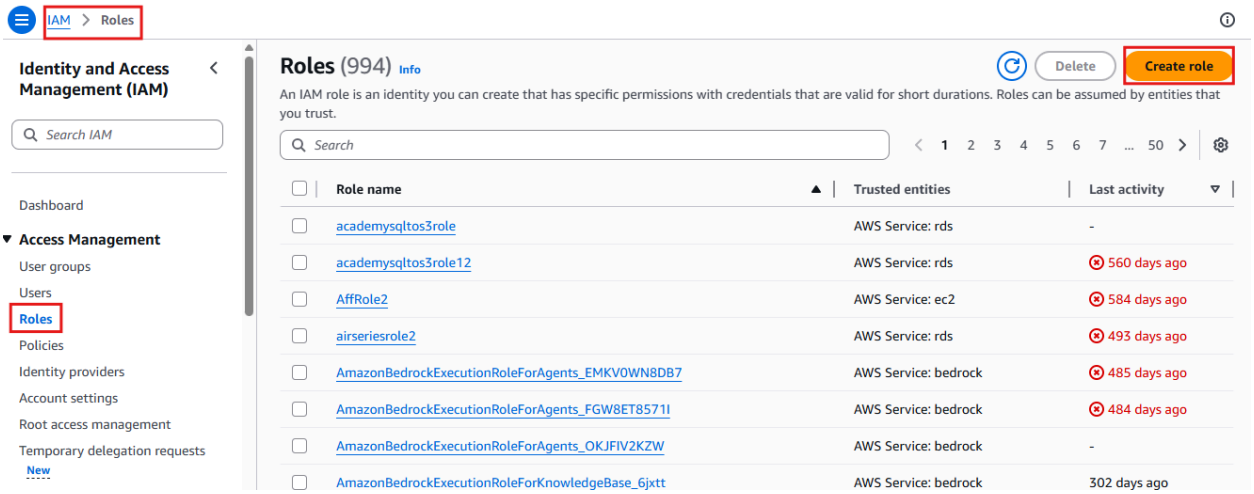
AWS Console → Glue → Confirm job



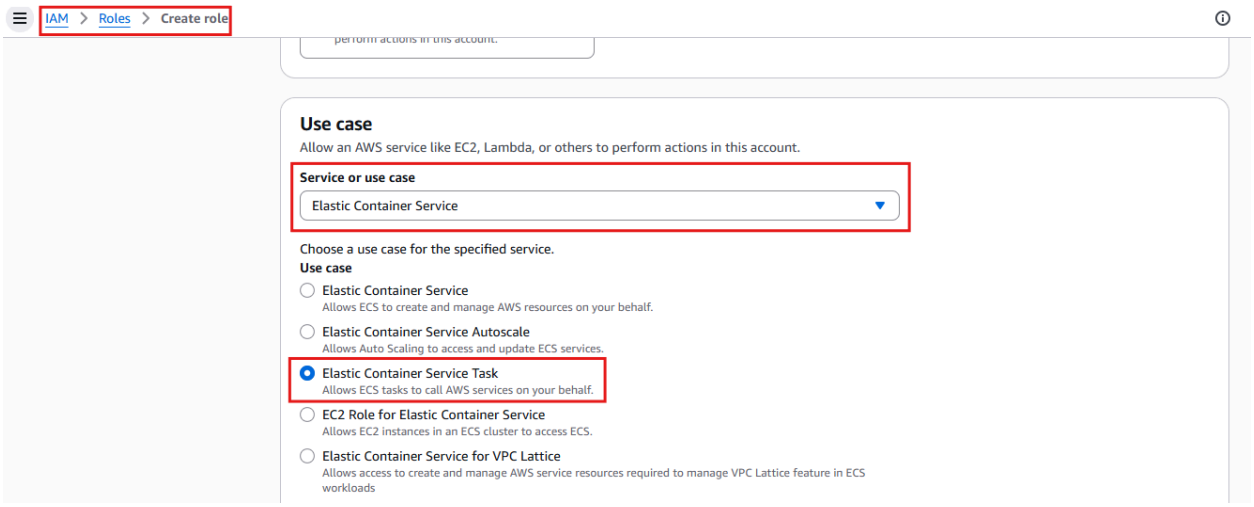
Task 6 — Deploy ECS Task Using CI/CD

Step 1 — Create IAM Role

AWS Console → IAM → Roles → Create Role



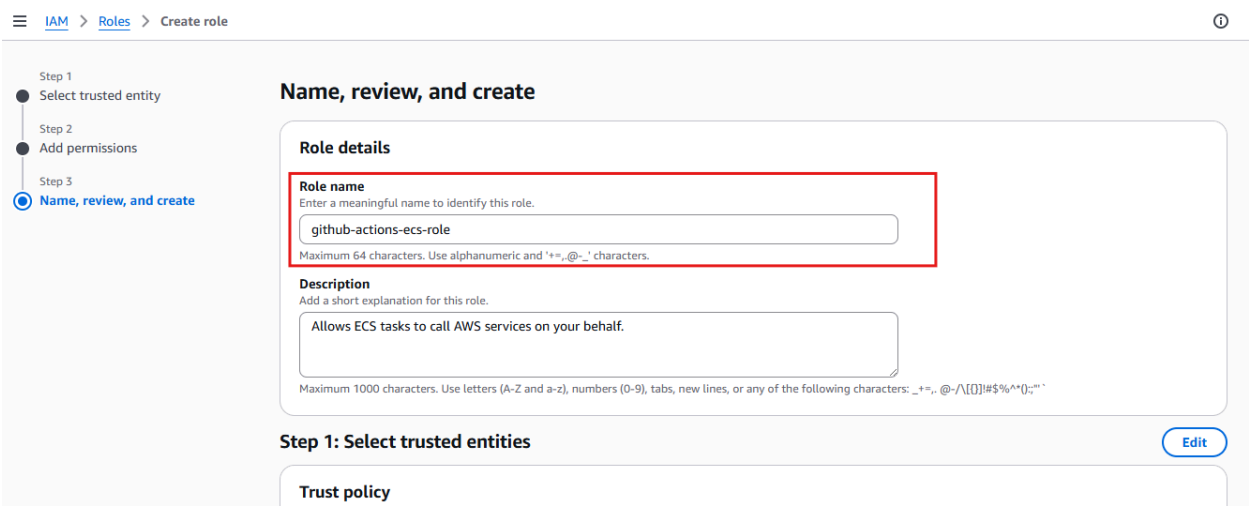
Select use case: ECS —→ ECS Service Task



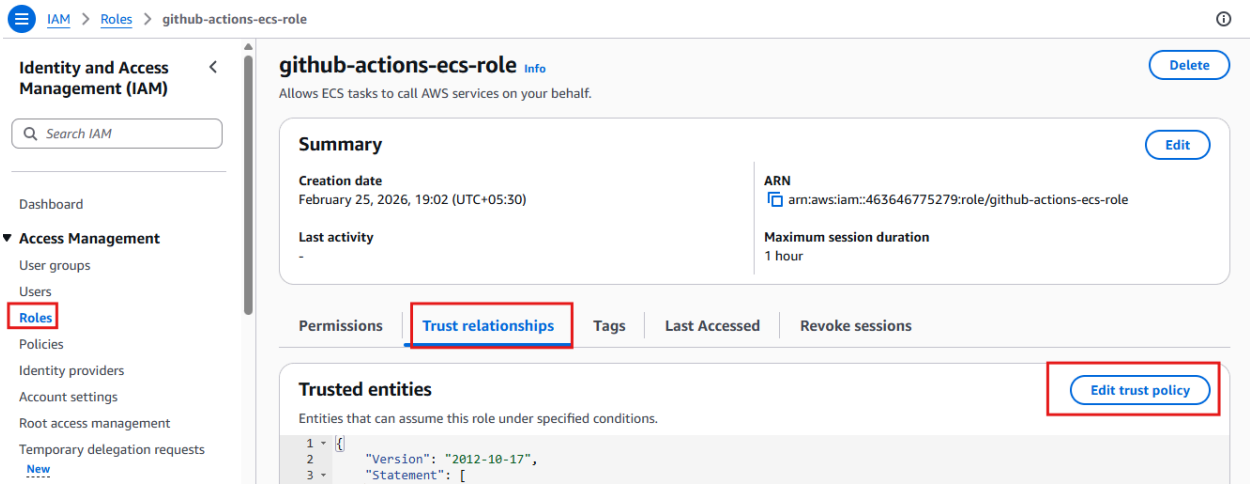
Add permissions:

ECSFullAccess
S3FullAccess
CloudWatchLogsFullAccess

Name the role —- Click create role



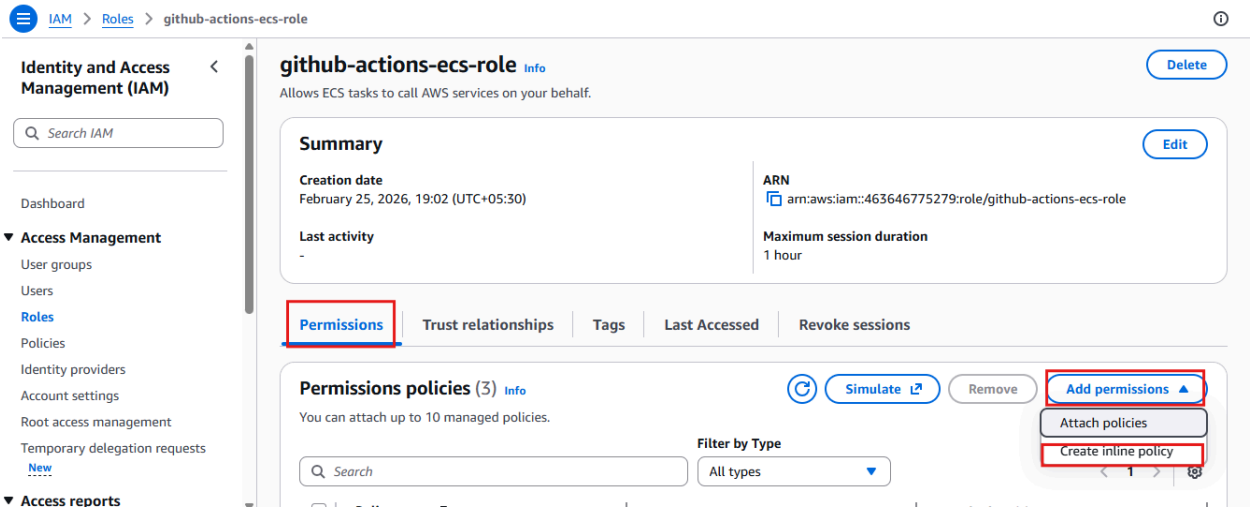
Click view role —→ Click on Trust relationship —→ Click on Edit trust policy



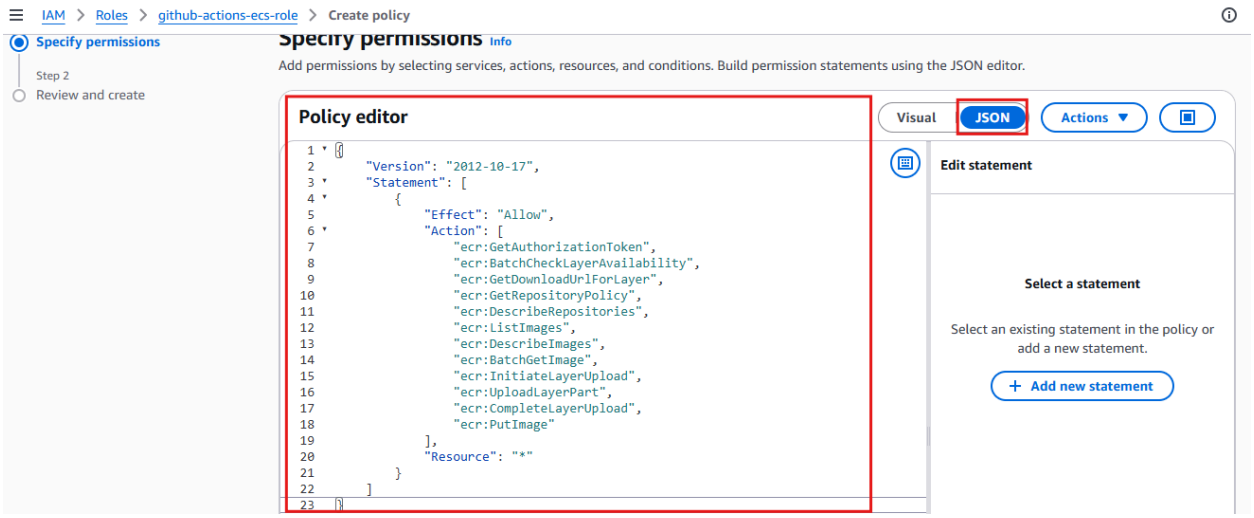
Replace with *iam_trust_relationship.json* code

Update account number and GitHub username —→ Click update policy

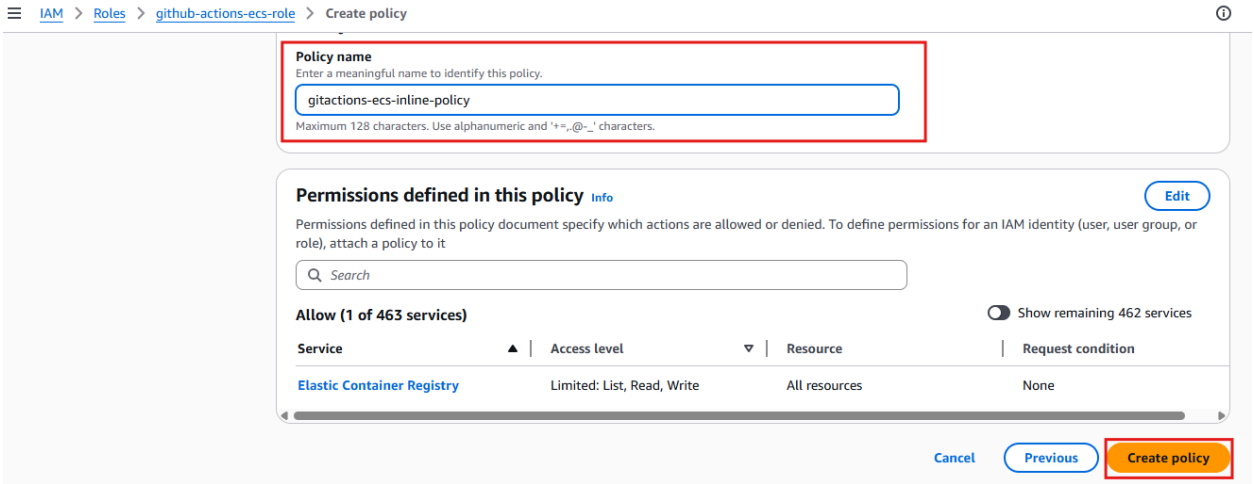
Add permission —→ create inline policy



Replace code from *gitaction-inline-policy.json*

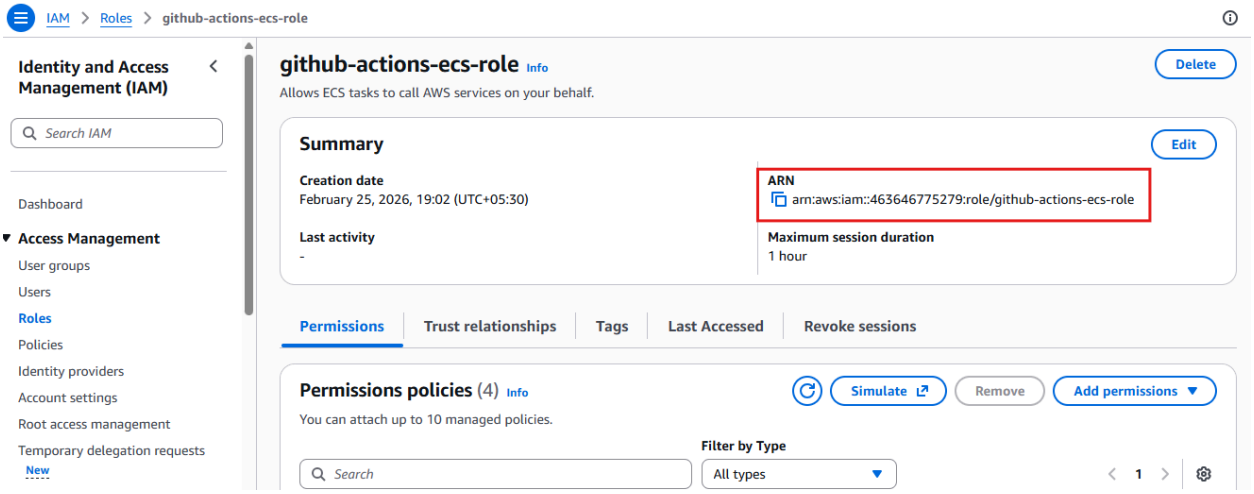


Name the policy —→ Scroll down —→ Click Create policy



Step 2 — Copy Role ARN

Copy the role ARN —→ Paste into workflow.yml file for github-actions-role ARN



Go to workflows file *role-to-assume:* Paste the copied ARN

Task 7 — Create GitHub Repository

Step 1 — Create Repo

GitHub → New repository

Upload:

- app.py
- requirements.txt
- dockerfile
- task-definition.json
- test_app.py

Step 2 — Add Workflow Directory

Create path:

```
.github/workflows/workflow.yml
```

Step 3 — Add GitHub Secrets

Go to Repository → Settings → Secrets → Actions

Add:

- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY

Step 4 — Add ECS Workflow YAML

.github/workflows/workflow.yml
Workflow should:

1️⃣ Build Docker image

2️⃣ Push to ECR

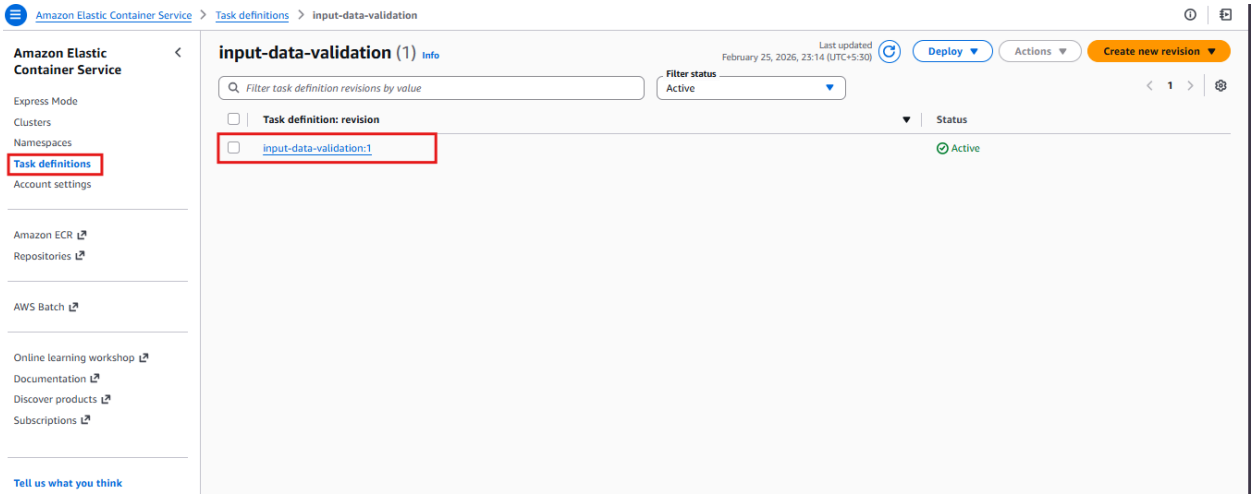
3️⃣ Deploy task

Step 5 — Commit Changes

Check GitHub Actions logs

Step 6 — Verify Deployment

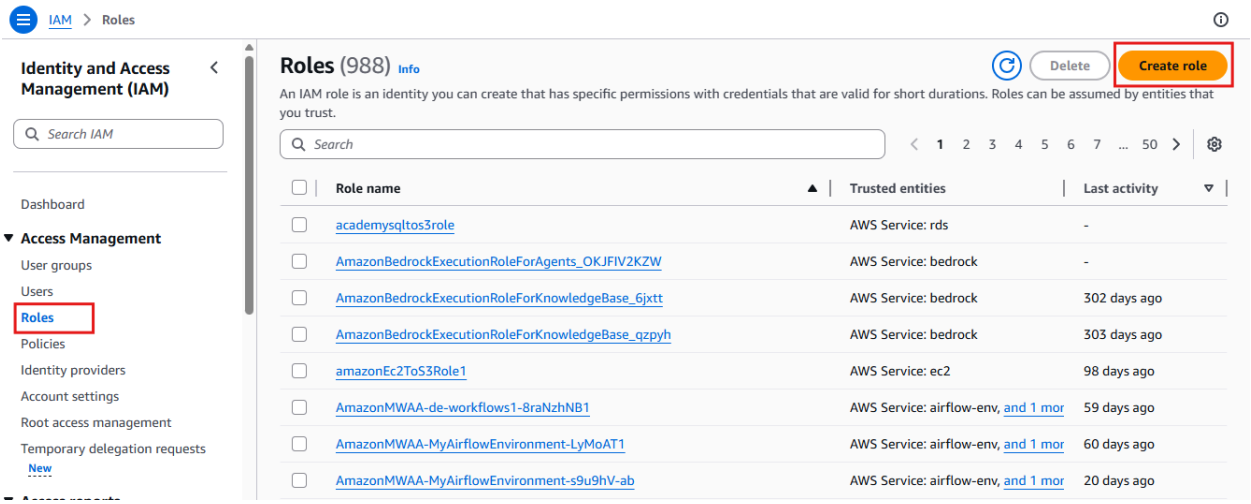
AWS Console → ECS → Task Definitions



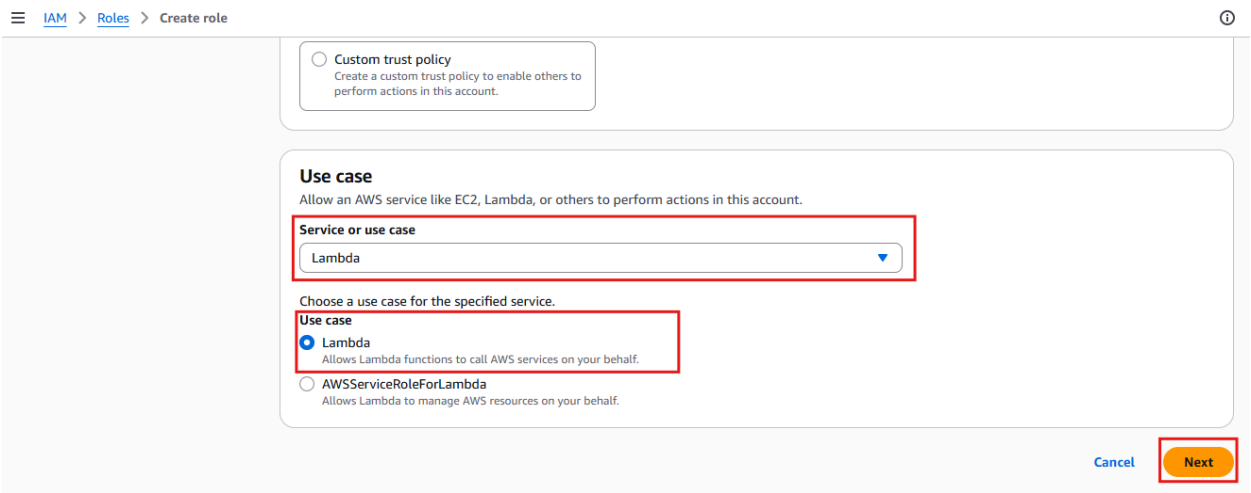
Task 8 — Deploy Lambda Using CI/CD

Step 1 — Create IAM role

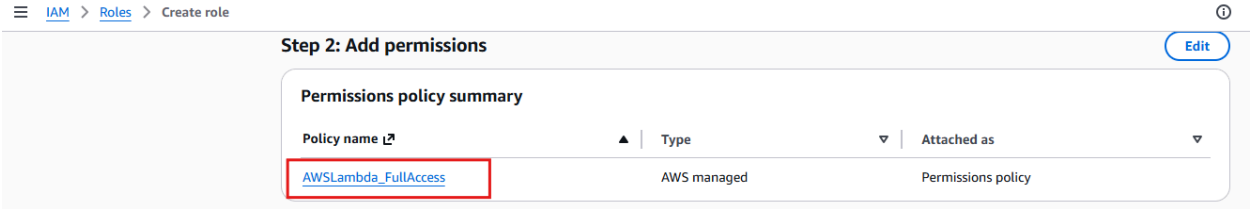
AWS Console → IAM → Roles → Create Role



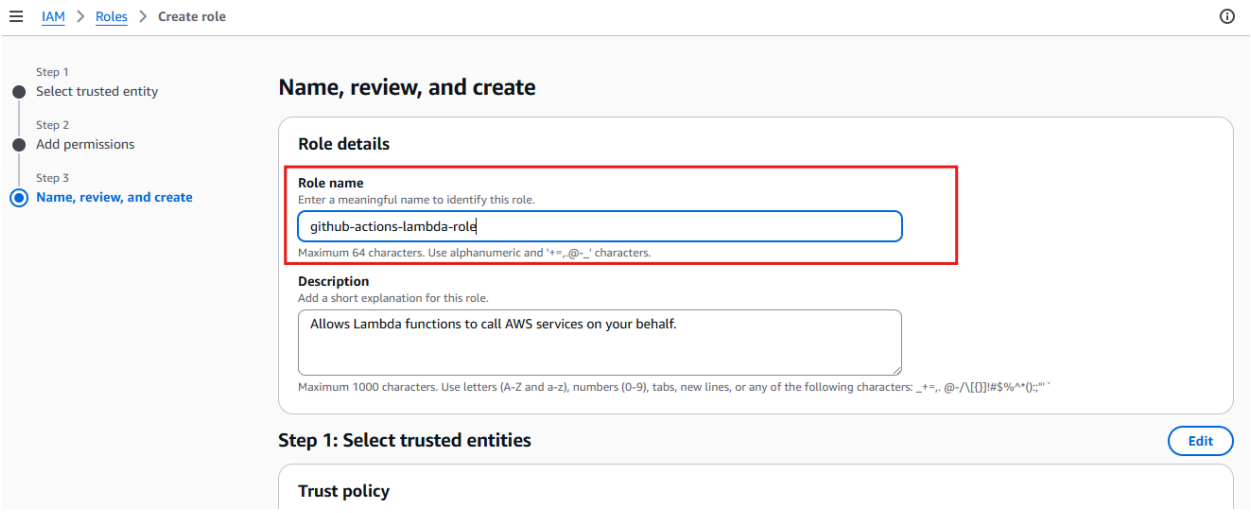
Select use case: *Lambda* —→ Click next



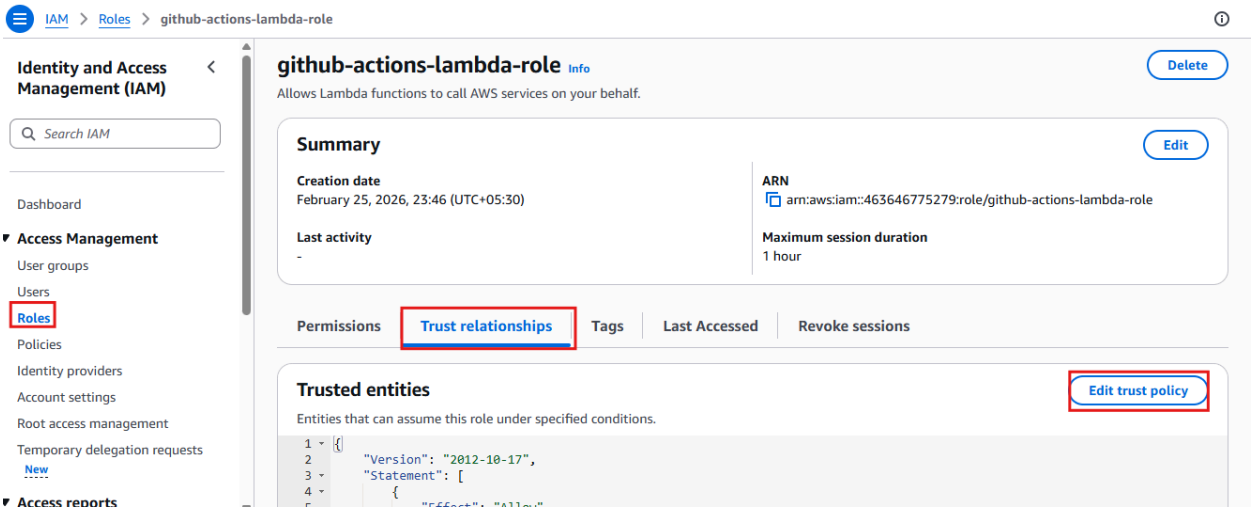
Add permissions: LambdaFullAccess



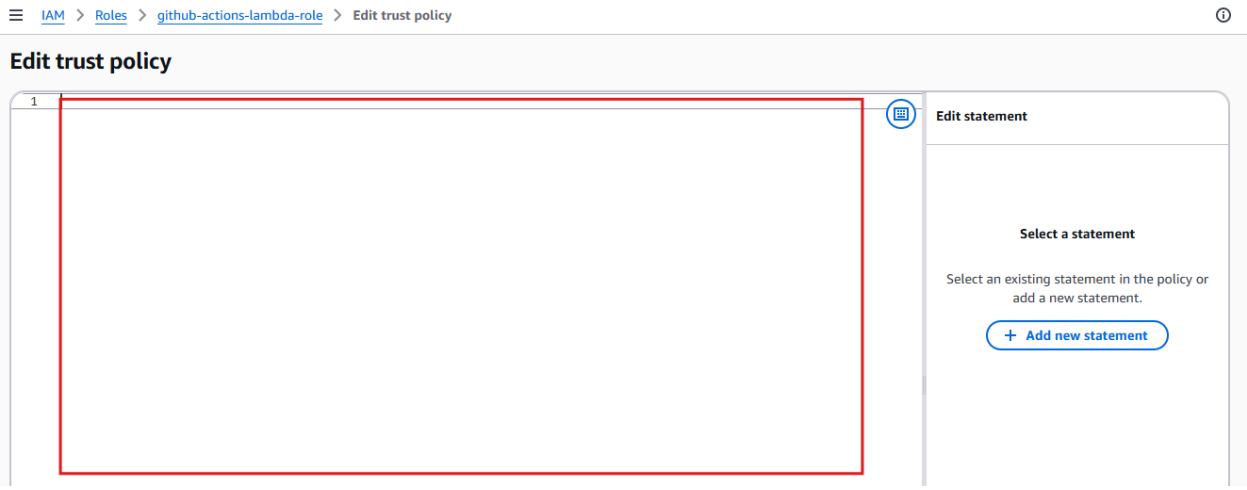
Name the role —- Click create role



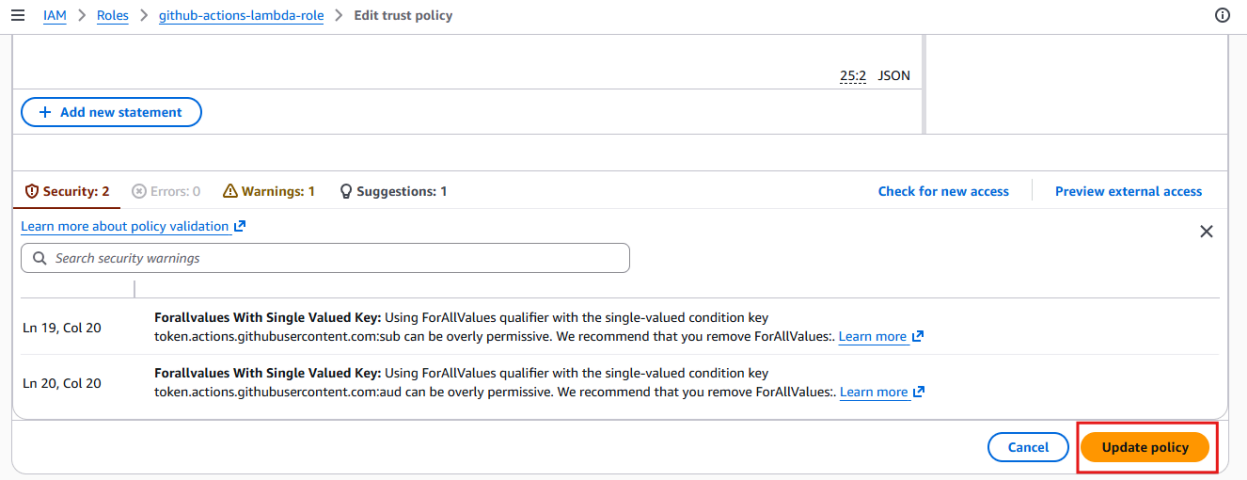
Click view role —→ Click on Trust relationship —→ Click on Edit trust policy



Replace with *lambda_iam_trust_relationship.json* code

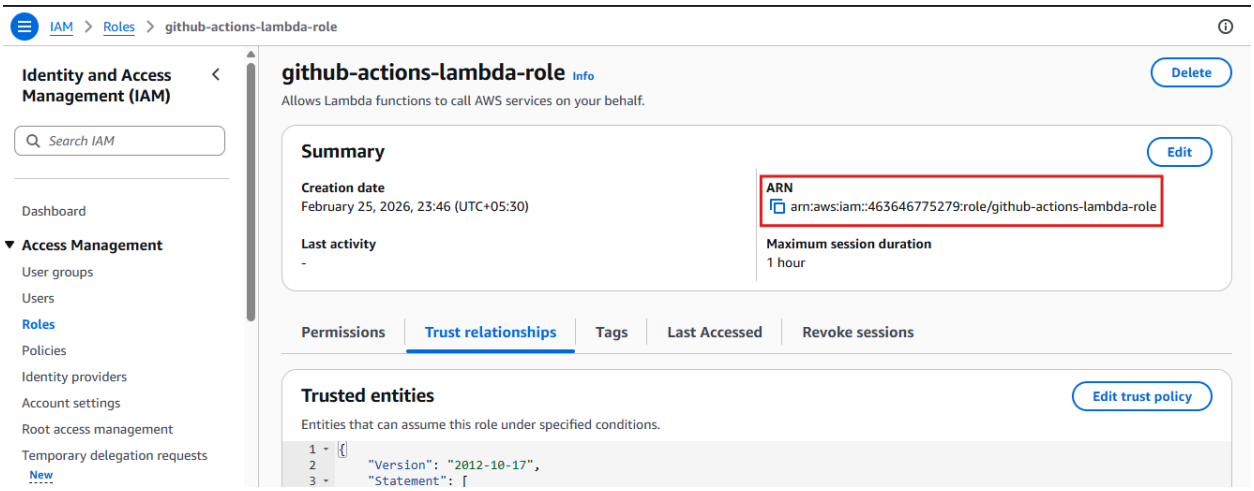


Update account number and GitHub username —→ Click update policy



Step 2 — Copy Role ARN

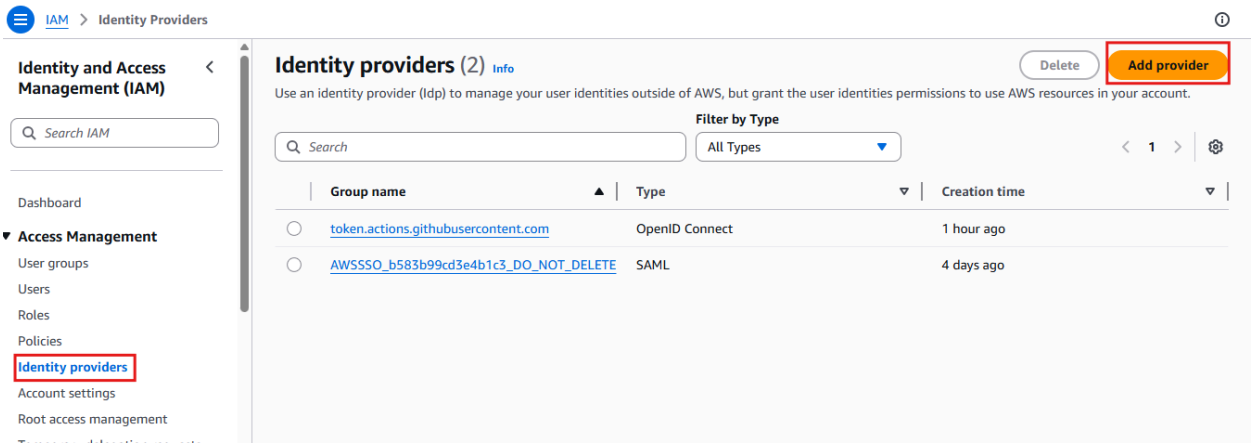
Copy the role ARN —→ Paste into workflow.yml file for github-actions-role ARN



Go to workflows file *role-to-assume:* Paste the copied ARN

Step 3 — Add Identity Provider

Click on Identity Provider —→ Add provider

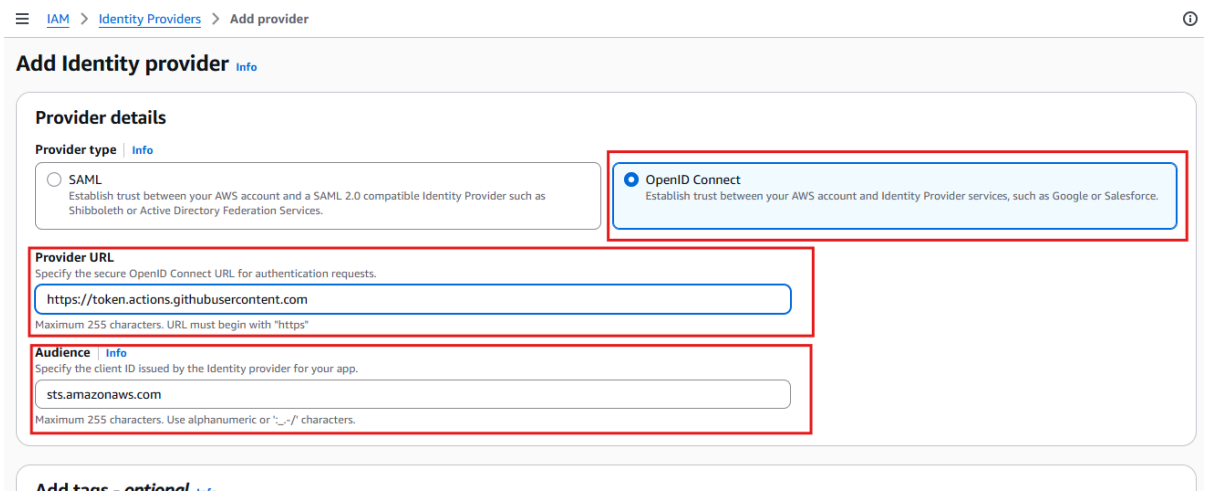


Add details:

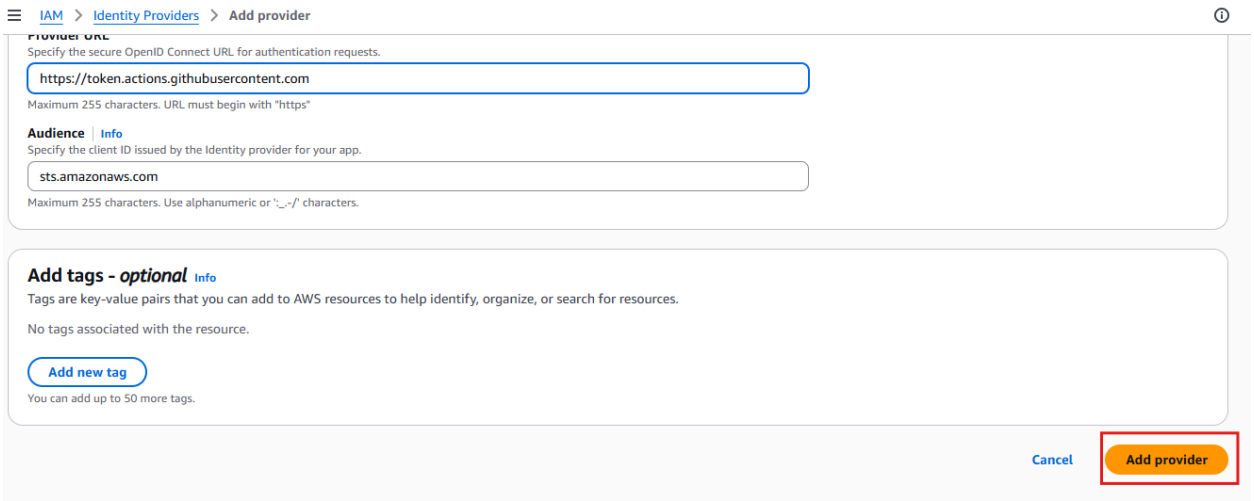
Provider type: OpenID Connect

Provider URL:https://token.actions.githubusercontent.com/

Audience: sts.amazonaws.com



Click Add Provider



Step 4 — Create Github Repository

GitHub → New repository —→ lambda-udemy-cicd

Step 5 — Upload Lambda Files

Upload:

- lambda_function.py
- test_lambda.py

Step 6 — Add Workflow YAML

.github/workflows/workflow.yml

Workflow should:

1️⃣ Install dependencies

2️⃣ Run tests

3️⃣ Zip function

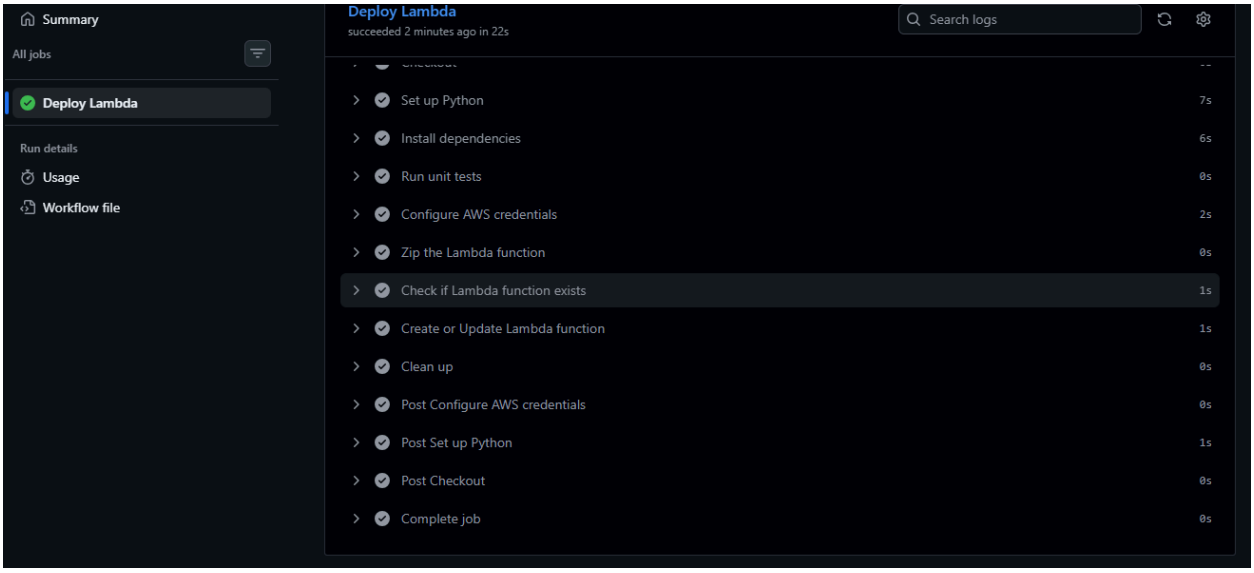
4️⃣ Deploy Lambda

Step 7 — Add Secrets

Add AWS keys in repository secrets

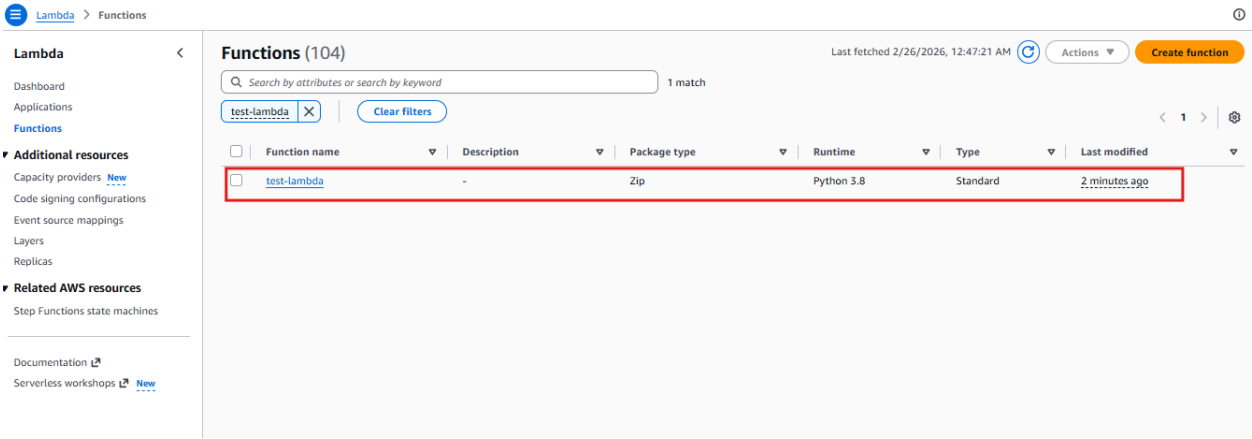
Step 8 — Commit Workflow

Check GitHub Actions logs



Step 9 — Verify Lambda

AWS Console → Lambda → Confirm function



Outcome

After completing this lab you will be able to:

- Deploy Lambda using CLI
- Deploy Glue jobs using CLI
- Configure GitHub Actions workflows
- Automate deployments using CI/CD
- Deploy ECS tasks via pipeline
- Use IAM roles for GitHub authentication
- Validate deployments using GitHub logs